

07/08/2026

Alexandra – a look-ahead SAT solver for And-Inverter graphs

by Andrei Oleynikov

Table of Contents

The Problem.....	3
Literature Review.....	3
The Method.....	4
The Algorithm: Consensus Driven Reverse Propagation (CDRP).....	4
Implementation: Alexandra.....	10
Evaluation.....	11
Dynamic cast.....	11
Deadlocks in the Thread Pool.....	11
Latch Propagation.....	12
Empty Handlers.....	12
Scheduling of Fold and Add tasks.....	12
Repeated branch visits.....	12
Memory leaks.....	12
Pulse structure and comparison.....	13
Pulse addition logic.....	13
Conclusion.....	13
Appendix.....	13
main.cpp.....	13
Solver.cpp.....	16
graph.h.....	27
UnitPulse.cpp.....	35
Stack.cpp.....	39
solver.h.....	40
Pulse.cpp.....	42
NTrue.cpp.....	58
NFalse.cpp.....	58
IPulse.cpp.....	59
AggregatedPulse.cpp.....	60
Utilities.h.....	71
ThreadPool.cpp.....	74
NAnd.cpp.....	77
NNot.cpp.....	78
NLatch.cpp.....	79
NVariable.cpp.....	80
Node.cpp.....	81
Propagate.cpp.....	81
StartPoint.cpp.....	82
NodeHandler.cpp.....	83
Accumulator.cpp.....	84
References and Acknowledgements.....	92

The Problem

Boolean Satisfiability Problem (or SAT for short) is a mathematical problem of identifying whether or not a given boolean function can return True. Being an NP-complete, it does not have a solution yet, however there are heuristic approaches to solving it in specific cases [1].

The practical applications of SAT are numerous: Boolean satisfiability can be used in process calculus to model potential deadlocks in concurrent systems [1]; SAT sweeping is a technique of electronic circuit optimisation, when SAT solver is used to identify logically equivalent branches on a circuit which are then merged into one [8][9][10]; By extending a solver with exclusive or function, it can be used to determine how difficult it is to decode a given encryption [5].

The most popular approach is called Conflict-Driven Clause Learning (CDCL)[3], is designed to work on functions in Conjunctive-Normal Form (CNF). This means that a problem can be represented as a set of independent OR clauses with up to three operands, all of which need to return True [2].

However, there are other forms of representation, for example And-Inverter Graph (AIG) [4]. This notation describes a problem as a set of logical gates (AND and NOT) and input/output variables. This form is more appropriate for circuit verification and optimisation [8][10], but there are not many solvers that work on it directly because there is a number of existing CNF-based have been developed for a really long time and so there are plenty very good implementations. Therefore, problems in AIG format are either translated into CNF [9] or solved using an adapted version of CDCL [8][10].

This paper will describe a new algorithm for SAT solving on And-Inverter graphs, based on AIGER format [4] and discuss the details of its implementation, in particular its design issues and logic errors.

Literature Review

Conflict-Driven Clause Learning is an algorithm that builds up on Davis-Putnam-Logemann-Loveland algorithm (DPLL). The way how they both work is through variable propagation and backtracking. [2][3]

In CNF problem is defined as a set of independent clauses. Each clause can have up to three variables (or negations of the variables). A clause is an OR function, so it is satisfied by at least one variable being true. Since variables are shared between different clauses, if a variable in one clause set to a value, it is supposed to be set to the same value in all clauses (which is known as Variable Propagation). [4][11]

In order to show that a problem is satisfiable or not, both CDCL and DPLL are trying to find a minimum satisfying configuration for all given clauses, while also propagating variables. As mentioned above clauses need at least one variable/negation set to true. If variables in a clause are unknown or false, the last variable must be set to true. Therefore, if variable is not the last or if there already variable(s) set to true, the variable in question is set to false. [2][3]

In both cases, solver keeps track of what clauses they visited and what values were assigned to variables. This is done so that solver can see contradictions (if a solver tries to set a variable to both

07/08/2026

true and false, commonly referred to as conflict) in their justification and revert to previous clause to try different value. [2][3]

The difference between algorithms is that DPLL will backtrack to the decision one clause back to the current clause, and will keep going up until it either resolves the conflict or there will be no more alternative decisions (if problem is not satisfiable). [2] CDCL on the other hand will note the configuration of the variables that led to the conflict, turn it into a new clause, add it to the problem, and backtrack to the earliest decision before any of the variables in the new clause were visited. This allows to minimise backtracking (as the solver knows how far it needs to jump) and ensures that the solver will not create the same conflicts. [3]

This method works well on CNF problems and have been used in many state-of-the-art solvers, such as MiniSAT and Glucose. Because of this, software for circuit verification and logic synthesis often uses external SAT engines for verification and sweeping, which means that a model of the circuit (which is already an AIG) needs to be translated into CNF. [7][8][9][10]

There are attempts to create an AIG native solvers, but they still rely on CDCL as the most effective known algorithm. The problem is that in logic networks the clauses and variables are somewhat equivalent, meaning that a clause can feed its output (or its negation) into another clause. This means that values of clauses also needs to be propagated, however it is possible only when a clause receives all inputs (which is referred to as justification). This leads to creation of justification frontier (j-frontier for short) - a set of all nodes on a graph that have not received all their inputs. This means that the solver needs to backtrack in order to justify nodes in the frontier (which can be a bottle neck for a performance), but it also creates a short cut for proving that a circuit is not satisfiable, as if j-frontier can not be emptied any further then the problem can not return true. [8][10]

When a solver is used for SAT sweeping it is used along with a circuit simulation. This is when nodes in a given part of the graph receive a sequence of random inputs and return a sequence of corresponding outputs. This sequences are fed into the nodes connected to the first ones. If the output sequences of the node are then compared. The nodes with equal output sequences are then categorised as equivalent. The SAT solver is then used to prove that these are equivalent and can be merged. The calls to SAT engine are the most time consuming, and so it is important to either have an efficient engine and/or a simulation method that can find potentially equivalent branches, while ignoring the ones that can be easily disproved. [8][9][10]

The Method

The Algorithm: Consensus Driven Reverse Propagation (CDRP)

The proposed algorithm is partially inspired by CDCL and circuit simulation. The idea is to re-imagine the concept of variable propagation on AIGs and enhance time performance by introducing parallelism into solving.

Let us consider logic gates AND and NAND. For whatever number of inputs (fanins) an output (fanout) of an AND gate can only be true if all fanins are true. Similarly, a NAND returns true

07/08/2026

whenever at least one fanin is false. Knowing the properties of the gates forming the problem and that a satisfiable problem can return true, it is possible to work out the satisfying input values (and even an underlying pattern for all satisfying inputs) directly by propagating an output value.

The steps for CDRP algorithm are as follows:

aag 5 2 0 1 3

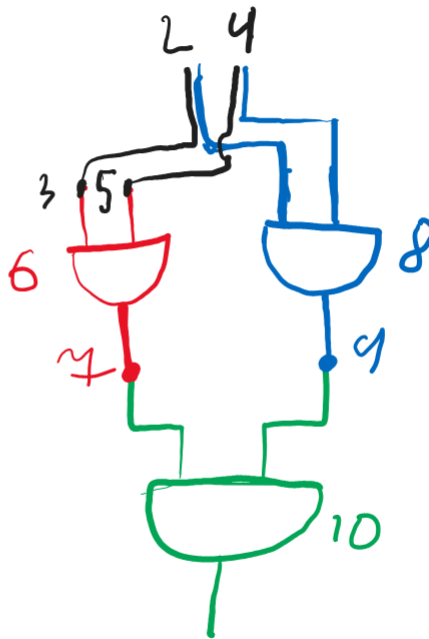
2

4

6 3 5

8 2 4

10 7 9



- Start at the fanout of the output node of the problem;

07/08/2026

aag52013

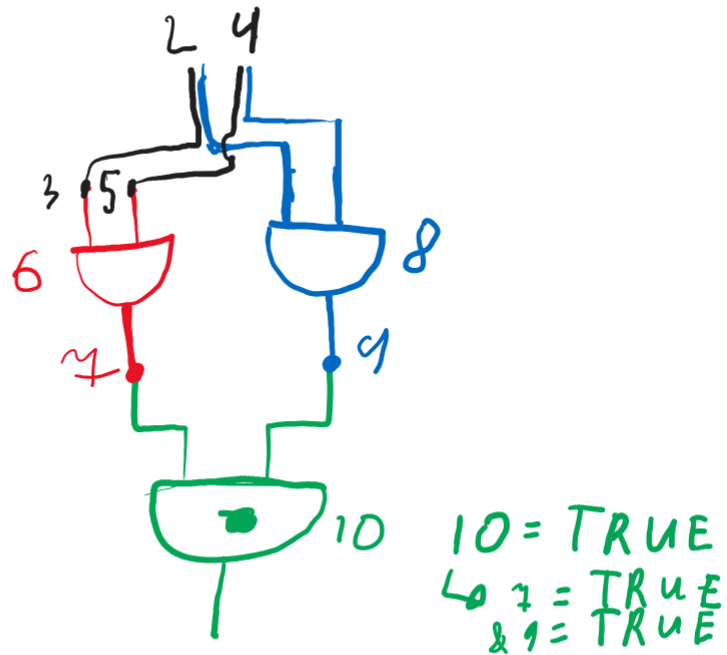
2

4

635

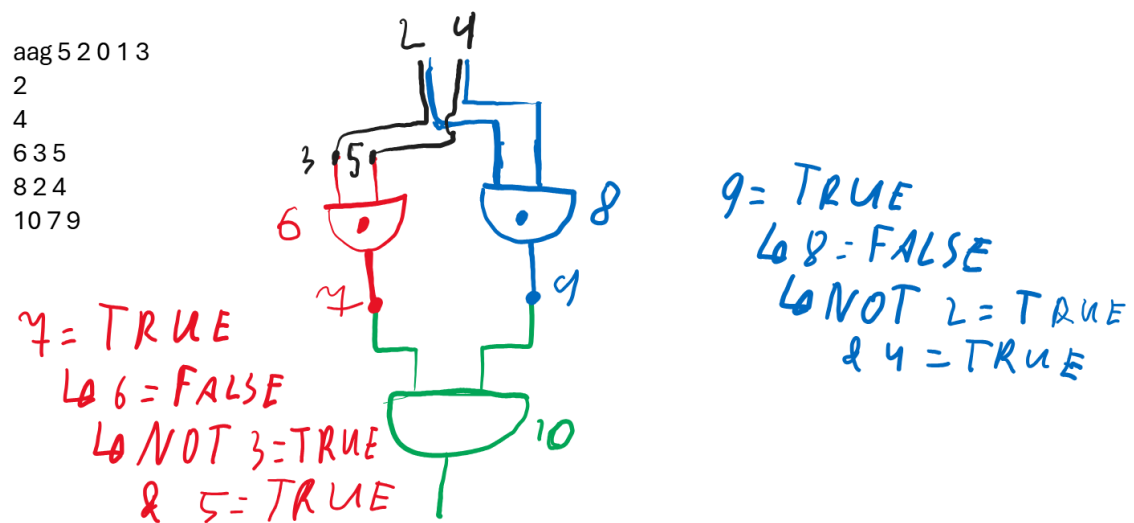
824

1079

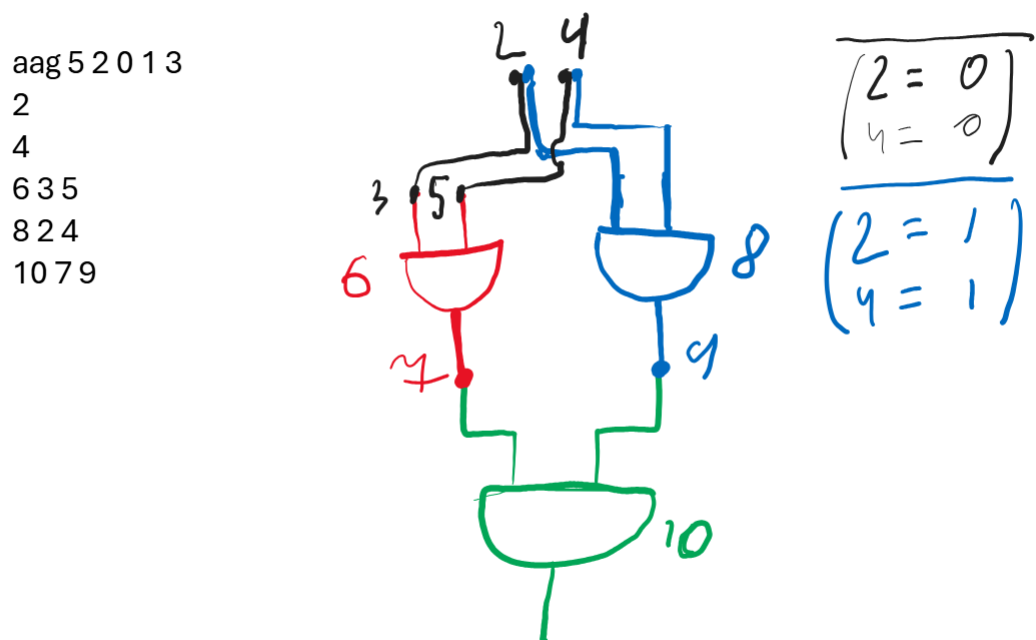


- Send a signal through the fanout of the node;
 - The signal should contain a list of NOT gates that it went through in reverse chronological order (the SAT trace);
 - A signal is assumed to be positive – this means that a signal is an output true for a given gate;
- Once the signal reaches a node it should propagate through its fanins into the next node;
 - If the node is an AND gate, then signal splits in two, which simultaneously travel to their respective fanins, containing exactly same traces;
 - If the node is an NOT gate, then the index of current node needs to be added to the signal's trace;

07/08/2026



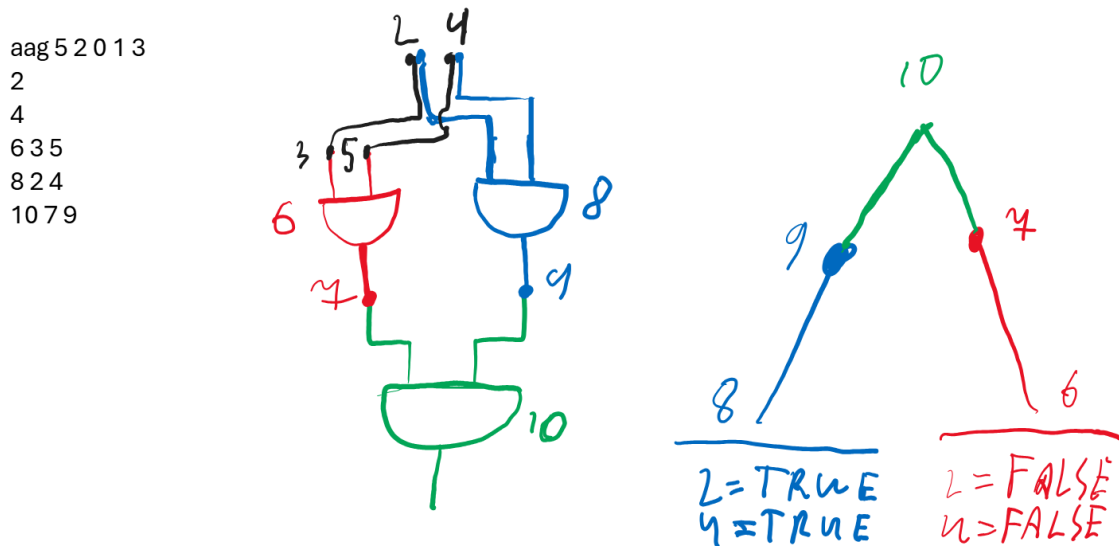
- Once signal reaches the variable node, its value must be added to a Semantic Tree;



- If signal's trace is empty, then variable is set to true and added into a root node;
- If signal's trace contains NOT gate who's fanin is the variable, then variable is set to false and the NOT is removed from trace;

07/08/2026

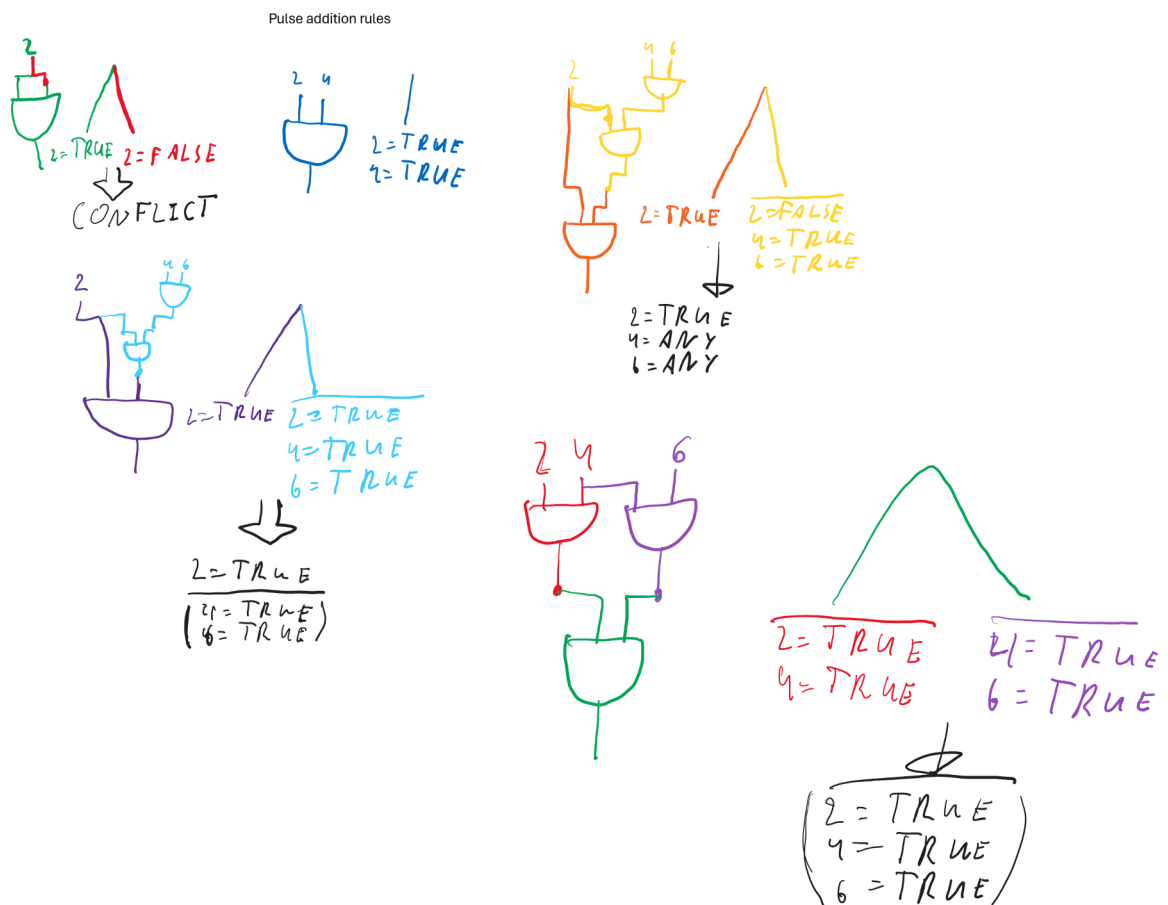
- While trace is not empty, create a child node (if does not exists already), add variable value to it or previous child (unless already added), remove last NOT gate from trace, if the trace is empty add current child to root node (unless already added);
- If a node contains a variable index/value pair and a new pair with the same index and opposite value is added, node contains a Conflict Pulse;



- Once all signals are added, merge nodes of the Semantic Tree together such that:
 - Index/value pairs in the node must form a Pulse – a pattern that describes all the satisfying solutions to a given branch/graph;
 - Master Pulse is considered positive, meaning it contains a satisfying input, and is formed from variables in the root node;
 - Child Pulses are considered negative, meaning that a satisfying input is any other then the one in the pulse, and are formed from child nodes;
 - Pulses can contain other pulses and lose index/value pairs;
 - Every pulse must contain unique variable/value pair (such that no variable index repeats);
 - Pulse from a child node must be added into a pulse from its parent.
 - If two pulses A and B stored together have common variable indexes, they are merged such that:
 - if common variables are set to the same values, added variables that are not present in unique variables from B into A;
 - otherwise, bundle pulses together into an Aggregated Pulse, as they can both be satisfied;

07/08/2026

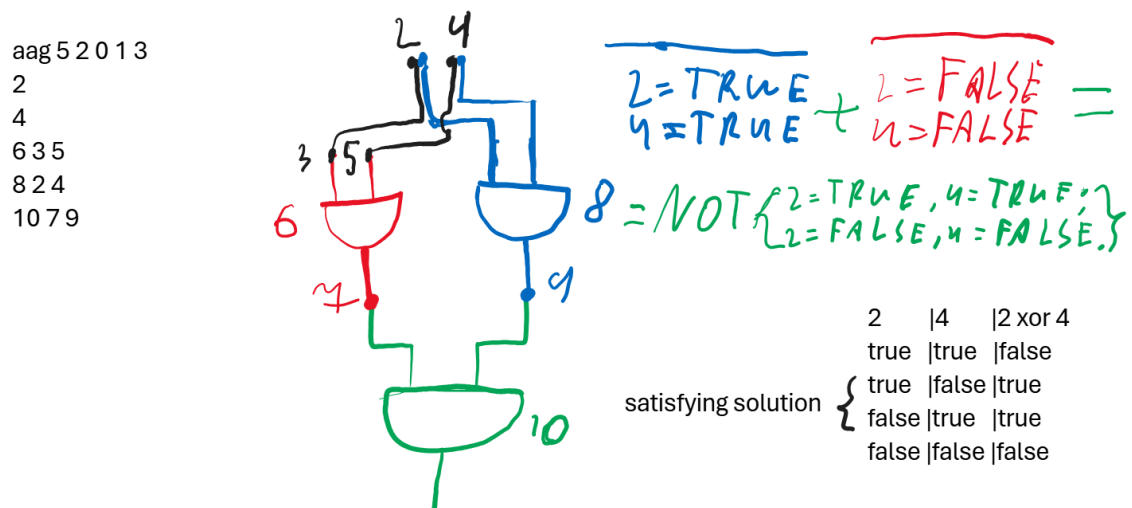
- If a Child Pulse is added into Master Pulse, and a Master contains a variable that is present in a Child then:
 - if a value of a variable is the same in both, remove variable from Child;
 - otherwise, remove variable from Child and mark it as satisfied, meaning that variables inside of it can be of any values;
 - store Child in a Master;
 - if a Child and a Master have all the same variables set to the same values, then conflict occurs;
- If a Child Pulse A is added into another Child Pulse B, contents of B are moved into A;
- If an Aggregate is added into any pulse it is either stored inside the pulse or merged if common variables present;
 - When merged with Master Pulse, all pulses stored in Aggregate are merged with Master, potentially forming a new Aggregate inside;
 - Aggregate cannot store another aggregate, they must be bundled into a pulse or their contents must form a new Aggregate (if common variables are present);



Once all nodes in Semantic Tree are merged, return final pulse;

07/08/2026

- If a pulse is a Conflict Pulse, problem cannot be satisfied;
- else, problem is satisfiable.



Pseudocode

Obj Node

```
{  
    • def type as either AND, VARIABLE, LATCH or NOT;  
    • def integer id;  
    • boolean watched;  
    • def Node[] next_node;  
    • new Node(id → input_id, watched → input_watched type → input_type, next_node →  
      input_nodes);  
}
```

Obj Accumulator

```
{  
    • Obj Accumulator_Node  
    • {  
        ◦ def boolean is_ripe → false;  
        ◦ def Pulse value;  
        ◦ def Accumulator_Node next_node;
```

07/08/2026

```

    ◦ new Accumulator_Node(id → input_id, optional next_node → input_next);
  • }
  • def Accumulator_Node[] nodes;
}
Interface IPulse;
Obj Unit_Pulse : IPulse
{
  • def state value → either TRUE, FALSE, ANY or CONFLICT;
  • def integer variable;
  • new Unit_Pulse(value → input_value, variable → input_variable);
}
Obj Pulse : IPulse
{
  • def boolean negative;
  • def UnitPulse[] values;
  • def Pulse[] pulses;
  • new Pulse(negative → is_negative, values → input_values[], pulses → input_pulses[]);
}
Obj Aggregated_Pulse : IPulse
{
  • def boolean negative;
  • def integer[] variables;
  • def Pulse[] entries;
  • new Aggregated_Pulse(negative → is_negative, entries → {...}, variables → input_variables[]);
}
Func result(Accumulator acc)
  • return pulse from AccNodes[0] in acc;
end function
Operator [](Accumulator acc, integer id)
  • if nodes in acc contain Accumulator_Node node with id = id then:
```

07/08/2026

- return node;
- else;
 - add new Accumulator_Node (id) to nodes;
 - return nodes[id];
- end if;

end operator

Operator + (IPulse left, IPulse right)

- //see next section

end operator

Meth propagate (Node node, stack sat_trace, latch_trace, Accumulator accumulator)

- switch type of node:
 - type is AND
 - {
 - if node is watched then:
 - put node id into sat_trace;
 - end if;
 - for each Node fanin in next_nodes:
 - branch off and propagate(fanin, sat_trace, latch_trace);
 - end for;
 - merge branches;
 - }
 - type is NOT
 - {
 - put node id into sat_trace
 - propagate next_nodes[0];
 - }
 - type is LATCH
 - {
 - if id of node is in latch_trace then:
 - accumulate (id, sat_trace, accumulator);

07/08/2026

- else:
 - put id of node into latch_trace;
 - propagate next_nodes[0];
- end if;
- }
- type is VARIABLE
- {
 - accumulate (id, sat_trace, accumulator);
- }
- end switch;

end method

Meth accumulate (integer variable, stack trace, Accumulator accumulator)

- def state value → either TRUE, FALSE, ANY or CONFLICT;
- if top value of trace = id + 1 then:
 - state → FALSE;
- else:
 - state → TRUE;
- end if;
- pop from trace;
- def Accumulator_Node acc_node → accumulator[top of trace trace];
- add new Unit_Pulse (value, variable) to acc_node;
- while trace is not empty:
 - pop trace;
 - add accumulator[top of trace] as next_node in acc_node;
 - acc_node → accumulator[top of trace] ;
- end while;

end method

Meth fold(Accumulator_node node)

- add pulse in node to a pulse in node's next_node;
- set is_ripe in next_node to true;

07/08/2026

end method

Begin:

- def Node node → get Node from Graph;
- def Accumulator accumulator → new Accumulator;
- def stack sat_trace;
- def stack latch_trace;
- while next_node in node is not empty:
 - propagate(node, sat_trace, latch_trace, accumulator);
- end while;
- def boolean complete → false;
- while (!complete):
 - for each Accumulator_Node accNode in accumulator such that accNode is ripe → true:
 - fold(accNode);
 - if accumulator is empty then:
 - complete → true;
 - end if;
 - end for;
- end while;
- output Pulse result from accumulator;

End.

Implementation: Alexandra

The solver is written in C++ programming language for better performance. It is a command line application that takes the following inputs: number of threads (unsigned integer), path to an .aag file (a file in an ASCII AIGER format), and an assumption (a sequence of variable indexes in a format similar to AIGER where even number is variable set to true and odd is variable set to false).

The solution is organised into three namespaces:

- Alexandra – main namespace, contains Solver and Result classes;
- Utilities – contains thread pool, a single-linked tree stack implementation for trace and unique error classes;
- Graph – contains classes that form And-Inverter Graph model, Pulses, and an Accumulator class (which implements Semantic Tree).

07/08/2026

When program is executed, it will parse the file, forming a syntax tree (And Inverter Graph). The parser is implemented as part of the Solver object's constructor. If the file does not have .aag ending or it doesn't follow AIGER format, parser will return invalid input error. The graph is stored as a map of node handlers (since clauses can be defined in any order it is necessary to create handlers which will then contain a pointer to a node), a pointer to an Accumulator object and an instance of a Start Point object, which contains pointers to the output node handlers (AIGER allows multiple nodes to be outputs). The parser can identify if a problem is satisfiable or not if its only output is set to 0 (always false) or 1 (always true), or not present at all. It will then through a respective error object from utility namespace.

The nodes store their own index and pointer(s) to their fanin clauses. There are four types of nodes: variable, AND, NOT, and a binary latch. A latch (flip-flop) node is part of AIGER format and it represents a storage mechanism (similar to real latches). It has a single fanout and a single fanin, which is a function that defines latch's next value. The variable nodes and latches store pointer to an Accumulator (which acts as both cache and semantic tree).

Once a graph has been formed, the Start Point object schedules propagation of the output nodes to the thread pool. During propagation, a signal consisting of two stacks, SAT trace and latch trace. The later one helps keep track of latches which were already visited for correct propagation. A vacant thread will pick up the task and start the process. Main thread will wait until propagation is complete. Every time a thread propagates the AND node it will schedule a new propagation task which will target a right fanin, while the thread will continue onto the left fanin.

When an NOT node or an AND node which occurs as fanin multiple times is encountered, it is added to a SAT trace. SAT trace is used to form semantic tree in accumulator in order to preserve context of the problem. The traces are implemented as a single-linked tree stacks, so that it can be easily copied and appended.

After propagation is done, Solver object will trigger a solve method in the Accumulator. Accumulator will schedule tasks for converting its nodes into pulses and then adding those pulses together.

There are three types of pulses:

- Unit Pulse – an index/value pair that can be true, false, any or conflicting. When added to aggregates or negative pulses it is considered positive;
- Pulse – a generic pulse that stores other pulse objects. Can be either positive or negative, but positive pulses cannot be stored;
- Aggregated Pulse – an object that stores a bundle of negative pulses with common indexes and opposite values. This is necessary to preserve relationships when there are multiple constraints that share variables but are not mutually exclusive. Can also be positive or negative, but considered to be negative during addition with positive pulses.

The pulses can be tested for equivalence (operator ==) or added (operator +). Two pulses are equivalent if they have unit pulses with common indexes. Addition operation either merges pulses or adds them into one another. The Result object in Alexandra namespace is wrapper for pulses that can be converted into a string, and is created once Accumulator finishes solving the semantic tree.

Evaluation

The Solver is capable of solving simple SAT problems (see appendix), but struggles with complex problems that involve multiple intertwined aggregations. In this situations its results are unreliable. The caching mechanism allows to only process branches ones, as well as removes unnecessary duplicates. The additions and propagation can be parallelized without sacrificing quality of results and gain boost in performance.

The only other issue other then flawed aggregate logic is memory management. The utility produces large amount of objects that are never used but also never get deleted. This can be solved by introducing copy constructors and assignment operations to pulse objects, thus allowing to create intermediate values on stack, rather then on heap.

Future improvements

- Tournament accumulator – right now accumulator functions as n-tree with random number of branches per node, which in cases of large graphs can lead to thundering herd problem, when loads of threads are trying to enter certain node simultaneously, thus stacking in a bottleneck. A proposed improvement to an accumulator would be to add binary tree underneath existing relationships, so that leaf nodes of the same branch can be processed in pairs, until they reach the parent value. This will allow to further capitalise on parallel capabilities;
- Modular parsers – although the algorithm was designed for AIG's, it can theoretically work on other representations, for example CNF. But creating an API for users to add external parsers and by extending graph namespace to accommodate other logical functions it will become universal SAT engine.
- Conflict short-circuiting – in current implementation conflict value absorbs all other values it added to, thus continuing propagation. Considering that in this algorithm conflicts are most likely unresolvable, it can be beneficial to halt entire propagation once conflict is identified and proven to be unresolvable.

Conclusion

The CDRP algorithm is functional method for solving sat that can provide much greater insight into a nature of given problem (for example it can simplify sat sweeping, as each branch can be analysed ones and their resultant pulses compared) and demonstrates that sat can be effectively parallelised. However, it still needs further improvements and developments

Appendix

Test.aag

07/08/2026

```
4 = TRUE is reported
new value is 4 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
6 = TRUE is reported to 2 = TRUE
new value is (2 = TRUE, 6 = TRUE)
6 = TRUE is reported
new value is 6 = TRUE
done
folding (2 = FALSE)
(2 = FALSE) is added
new value is (2 = FALSE)
folding (6 = FALSE)
(6 = FALSE) is added to (2 = FALSE)
new value is (6 = FALSE, 2 = FALSE)
folding Not (2 = TRUE, 6 = TRUE)
Not (2 = TRUE, 6 = TRUE) is added to 4 = TRUE
new value is (4 = TRUE, Not (2 = TRUE, 6 = TRUE))
folding Not (6 = FALSE, 2 = FALSE)
Not (6 = FALSE, 2 = FALSE) is added to (4 = TRUE, Not (2 = TRUE, 6 = TRUE))
new value is (4 = TRUE, Neither [(2 = TRUE, 6 = TRUE), (6 = FALSE, 2 = FALSE)])
folding (4 = TRUE, Neither [(2 = TRUE, 6 = TRUE), (6 = FALSE, 2 = FALSE)])
done
the result: (4 = TRUE, Neither [(2 = TRUE, 6 = TRUE), (6 = FALSE, 2 = FALSE)])
```

Test2.aag

```
2 = TRUE is reported
new value is 2 = TRUE
6 = TRUE is reported
new value is 6 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
4 = TRUE is reported
new value is 4 = TRUE
10 = TRUE is reported to 6 = TRUE
new value is (6 = TRUE, 10 = TRUE)
8 = TRUE is reported to (6 = TRUE, 10 = TRUE)
new value is (6 = TRUE, 10 = TRUE, 8 = TRUE)
4 = TRUE is reported
new value is 4 = TRUE
12 = TRUE is reported to (6 = TRUE, 10 = TRUE, 8 = TRUE)
new value is (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE)
done
folding (2 = FALSE)
(2 = FALSE) is added to 4 = TRUE
new value is (2 = FALSE, 4 = TRUE)
folding (4 = FALSE)
(4 = FALSE) is added to 2 = TRUE
new value is (4 = FALSE, 2 = TRUE)
folding Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE)
Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE) is added
new value is (Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
folding Not (2 = FALSE, 4 = TRUE)
Not (2 = FALSE, 4 = TRUE) is added to (Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
new value is (Not (2 = FALSE, 4 = TRUE), Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
folding Not (4 = FALSE, 2 = TRUE)
Not (4 = FALSE, 2 = TRUE) is added to (Not (2 = FALSE, 4 = TRUE), Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
new value is (Neither [(2 = FALSE, 4 = TRUE), (4 = FALSE, 2 = TRUE)], Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
folding (Neither [(2 = FALSE, 4 = TRUE), (4 = FALSE, 2 = TRUE)], Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
done
the result: (Neither [(2 = FALSE, 4 = TRUE), (4 = FALSE, 2 = TRUE)], Not (6 = TRUE, 10 = TRUE, 8 = TRUE, 12 = TRUE))
```

Test3.aag

07/08/2026

```
4 = TRUE is reported
new value is 4 = TRUE
4 = TRUE is reported
new value is 4 = TRUE
4 = TRUE is reported
new value is 4 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
2 = TRUE is reported to 4 = TRUE
new value is (4 = TRUE, 2 = TRUE)
done
folding (2 = FALSE)
(2 = FALSE) is added
new value is (2 = FALSE)
folding (4 = FALSE)
(4 = FALSE) is added to (2 = FALSE)
new value is (4 = FALSE, 2 = FALSE)
folding Not (4 = TRUE, 2 = TRUE)
Not (4 = TRUE, 2 = TRUE) is added
new value is (Not (4 = TRUE, 2 = TRUE))
folding Not (4 = FALSE, 2 = FALSE)
Not (4 = FALSE, 2 = FALSE) is added to (Not (4 = TRUE, 2 = TRUE))
new value is Neither [(4 = TRUE, 2 = TRUE), (4 = FALSE, 2 = FALSE)]
folding (Either [(4 = TRUE, 2 = TRUE), (4 = FALSE, 2 = FALSE)])
(Either [(4 = TRUE, 2 = TRUE), (4 = FALSE, 2 = FALSE)]) is added to 4 = TRUE
conflict no 2
new value is (2 = TRUE, 4 = TRUE)
folding (2 = TRUE, 4 = TRUE)
done
the result: (2 = TRUE, 4 = TRUE)
```

Test4.aag

```
2 = TRUE is reported
new value is 2 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
done
folding (2 = FALSE)
(2 = FALSE) is added to 2 = TRUE
conflict no 2
new value is 0 = CONFLICT
folding (0 = CONFLICT)
done
the result: 0 = CONFLICT
```

Test5.aag

07/08/2026

```
2 = TRUE is reported
new value is 2 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
6 = TRUE is reported to 2 = TRUE
new value is (2 = TRUE, 6 = TRUE)
4 = TRUE is reported to (2 = TRUE, 6 = TRUE)
new value is (2 = TRUE, 6 = TRUE, 4 = TRUE)
2 = TRUE is reported
new value is 2 = TRUE
6 = TRUE is reported
new value is 6 = TRUE
4 = TRUE is reported
new value is 4 = TRUE
4 = TRUE is reported
new value is 4 = TRUE
8 = TRUE is reported to (2 = TRUE, 6 = TRUE, 4 = TRUE)
new value is (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE)
6 = TRUE is reported to 2 = TRUE
new value is (2 = TRUE, 6 = TRUE)
8 = TRUE is reported to 4 = TRUE
new value is (4 = TRUE, 8 = TRUE)
8 = TRUE is reported
new value is 8 = TRUE
done
folding (2 = FALSE)
(2 = FALSE) is added to (4 = TRUE, 8 = TRUE)
new value is (2 = FALSE, 4 = TRUE, 8 = TRUE)
(2 = FALSE) is added
new value is (2 = FALSE)
folding (4 = FALSE)
(4 = FALSE) is added to (2 = FALSE)
new value is (4 = FALSE, 2 = FALSE)
(4 = FALSE) is added to (2 = TRUE, 6 = TRUE)
new value is (4 = FALSE, 2 = TRUE, 6 = TRUE)
folding (6 = FALSE)
(6 = FALSE) is added to (2 = FALSE, 4 = TRUE, 8 = TRUE)
new value is (6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE)
(6 = FALSE) is added to (4 = FALSE, 2 = FALSE)
new value is (6 = FALSE, 4 = FALSE, 2 = FALSE)
folding (8 = FALSE)
(8 = FALSE) is added to (6 = FALSE, 4 = FALSE, 2 = FALSE)
new value is (8 = FALSE, 6 = FALSE, 4 = FALSE, 2 = FALSE)
(8 = FALSE) is added to (4 = FALSE, 2 = TRUE, 6 = TRUE)
new value is (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)
folding Not (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE)
Not (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE) is added
new value is (Not (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE))
folding Not (6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE)
Not (6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE) is added
new value is (Not (6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE))
folding Not (8 = FALSE, 6 = FALSE, 4 = FALSE, 2 = FALSE)
Not (8 = FALSE, 6 = FALSE, 4 = FALSE, 2 = FALSE) is added to (Not (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE))
new value is Neither [(2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 6 = FALSE, 4 = FALSE, 2 = FALSE)]
folding Not (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)
Not (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE) is added to (Not (6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE))
new value is Neither [(6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)]
folding (Either [(6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)])
(Either [(6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)]) is added to Neither [(2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 6 = FALSE, 4 = FALSE, 2 = FALSE)]
new value is Either [(6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)]
folding (Either [(6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)])
done
the result: Either [(6 = FALSE, 2 = FALSE, 4 = TRUE, 8 = TRUE), (8 = FALSE, 4 = FALSE, 2 = TRUE, 6 = TRUE)]
```

Test6.aag

```
2 = TRUE is reported
new value is 2 = TRUE
10 = TRUE is reported
new value is 10 = TRUE
6 = TRUE is reported
new value is 6 = TRUE
4 = TRUE is reported to 10 = TRUE
new value is (10 = TRUE, 4 = TRUE)
2 = TRUE is reported
new value is 2 = TRUE
4 = TRUE is reported
new value is 4 = TRUE
12 = TRUE is reported to (10 = TRUE, 4 = TRUE)
new value is (10 = TRUE, 4 = TRUE, 12 = TRUE)
8 = TRUE is reported to (10 = TRUE, 4 = TRUE, 12 = TRUE)
new value is (10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)
6 = TRUE is reported
new value is 6 = TRUE
4 = TRUE is reported to 2 = TRUE
new value is (2 = TRUE, 4 = TRUE)
8 = TRUE is reported
new value is 8 = TRUE
8 = TRUE is reported to 6 = TRUE
new value is (6 = TRUE, 8 = TRUE)
done
folding (2 = FALSE)
(2 = FALSE) is added to (10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)
new value is (2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)
(2 = FALSE) is added
new value is (2 = FALSE)
folding (4 = FALSE)
(4 = FALSE) is added to (2 = FALSE)
new value is (4 = FALSE, 2 = FALSE)
folding (6 = FALSE)
(6 = FALSE) is added to (2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)
new value is (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)
(6 = FALSE) is added
new value is (6 = FALSE)
folding (8 = FALSE)
(8 = FALSE) is added to (6 = FALSE)
new value is (8 = FALSE, 6 = FALSE)
folding Not (2 = TRUE, 4 = TRUE)
Not (2 = TRUE, 4 = TRUE) is added
new value is (Not (2 = TRUE, 4 = TRUE))
folding Not (6 = TRUE, 8 = TRUE)
Not (6 = TRUE, 8 = TRUE) is added to (Not (2 = TRUE, 4 = TRUE))
new value is (Not (6 = TRUE, 8 = TRUE), Not (2 = TRUE, 4 = TRUE))
folding Not (4 = FALSE, 2 = FALSE)
Not (4 = FALSE, 2 = FALSE) is added to (Not (6 = TRUE, 8 = TRUE), Not (2 = TRUE, 4 = TRUE))
new value is (Not (6 = TRUE, 8 = TRUE), Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
folding Not (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)
Not (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE) is added to (Not (6 = TRUE, 8 = TRUE), Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
new value is (Neither [(2 = ANY, 10 = ANY, 4 = ANY, 12 = ANY, 6 = TRUE, 8 = TRUE), (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)], Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
folding Not (8 = FALSE, 6 = FALSE)
Not (8 = FALSE, 6 = FALSE) is added to (Neither [(2 = ANY, 10 = ANY, 4 = ANY, 12 = ANY, 6 = TRUE, 8 = TRUE), (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE)], Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
conflict no 2
conflict no 2
new value is (Neither [(2 = ANY, 10 = ANY, 4 = ANY, 12 = ANY, 6 = TRUE, 8 = TRUE), (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE), (8 = FALSE, 6 = FALSE)], Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
folding (Neither [(2 = ANY, 10 = ANY, 4 = ANY, 12 = ANY, 6 = TRUE, 8 = TRUE), (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE), (8 = FALSE, 6 = FALSE)], Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
done
the result: (Neither [(2 = ANY, 10 = ANY, 4 = ANY, 12 = ANY, 6 = TRUE, 8 = TRUE), (6 = FALSE, 2 = FALSE, 10 = TRUE, 4 = TRUE, 12 = TRUE, 8 = TRUE), (8 = FALSE, 6 = FALSE)], Neither [(2 = TRUE, 4 = TRUE), (4 = FALSE, 2 = FALSE)])
```

Test7.aag

07/08/2026

```
2 = TRUE is reported
new value is 2 = TRUE
2 = TRUE is reported
new value is 2 = TRUE
6 = TRUE is reported to 2 = TRUE
new value is (2 = TRUE, 6 = TRUE)
4 = TRUE is reported to (2 = TRUE, 6 = TRUE)
new value is (2 = TRUE, 6 = TRUE, 4 = TRUE)
6 = TRUE is reported to 2 = TRUE
new value is (2 = TRUE, 6 = TRUE)
4 = TRUE is reported to (2 = TRUE, 6 = TRUE)
new value is (2 = TRUE, 6 = TRUE, 4 = TRUE)
8 = TRUE is reported to (2 = TRUE, 6 = TRUE, 4 = TRUE)
new value is (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE)
8 = TRUE is reported to (2 = TRUE, 6 = TRUE, 4 = TRUE)
new value is (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE)
done
folding Not (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE)
Not (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE) is added to (2 = TRUE, 6 = TRUE, 4 = TRUE, 8 = TRUE)
conflict no 2
new value is 0 = CONFLICT
folding (0 = CONFLICT)
done
the result: 0 = CONFLICT
```

References and Acknowledgements

[1] Boolean Satisfiability Problem on Wikipedia

https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

[2] Davis-Putnam-Logemann-Loveland algorithm on Wikipedia

https://en.wikipedia.org/wiki/DPLL_algorithm

[3] Conflict-Driven Clause Learning on Wikipedia

https://en.wikipedia.org/wiki/Conflict-driven_clause_learning

[4] And-Inverter Graph on Wikipedia

https://en.wikipedia.org/wiki/And-inverter_graph

[5] AIGER format on GitHub – Armin Biere et. al.

<https://github.com/arminbiere/aiger/blob/master/FORMAT>

[6] “Extending SAT Solvers to Cryptographic Problems” Mate Soos et. al. (2009)

[7] “An Extensible SAT Solver” Niklas Een and Niklas Sörenson, version 1.2 (2003)

[8] “A Circuit-Based SAT Solver For Logic Synthesis” He-Teng Zhang et. al. (2021)

[9] “SAT-Sweeping Enhanced For Logic Synthesis” Luca Amaru et. al. (2020)

[10] “Deep Integration of Circuit Simulator and SAT Solver” He-Teng Zhang et. al. (2021)

[11] Conjunctive-Normal Form on Wikipedia

07/08/2026

https://en.wikipedia.org/wiki/Conjunctive_normal_form